

GimliForge v1.0

Solución a la creación de informes para pentesters
que no saben hacer informes

TRABAJO DE FIN DE MÁSTER: PARTE I

Autor: Adrián Suárez Recarey

Índice

- . 1. Introducción y motivación del proyecto.
- . 2. El flujo de datos
- . 3. Código, rigor y seguridad en la automatización
- . 4. Roadmap de Evolución (Futuras Mejoras)
- . 5. Anotaciones sobre el uso de la IA en este trabajo y otros detalles

1. Introducción y motivación del proyecto.

GimliForge (también abreviado como GIMLI) [Generador de Informes Mediante Lenguaje Inteligente] Nace de poner solución a una de las cosas que más me aterran a nivel personal. Tener que redactar un informe. Buscando que esta herramienta pueda ser de utilidad en el futuro he decidido crear esta pequeña utilidad, con intención de poder expandirla y que sea beneficiosa tanto para compañeros del Máster como para profesionales del sector.

¿Por qué LaTeX?

Muy sencillo, la publicación de papers relativos a la investigación suelen utilizar este lenguaje y aprovechando que hace poco hice un curso sobre ello, he decidido conectar los puntos y aplicarlo en esta tarea.

El nombre de Gimli proviene del señor de la saga de “El Señor de los Anillos” y he decidido rendir un pequeño homenaje al pequeño hombre barbudo y bajito con el que comparto ciertas cosas en común.

Es guerrero, artesano y un noble de la raza de los enanos, es por eso que he decidido que este proyecto se denomine Gimli Forge.

Para poder intentar explicar esta herramienta lo mejor posible lo que he hecho ha sido dividir el flujo de trabajo en 5 partes sencillas.

2. El flujo de datos

- **La Mina (Input):** Ingesta de datos bajo el estándar **G.I.M.L.I.** en formato JSON
- **El Bellows (Procesamiento):** Escrito en Python, utiliza la librería `readline` para facilitar la navegación y `glob` para el manejo de rutas.
- **El Cerebro IA (Traducción):** Integración con **Gemini 2.5 Flash** mediante la API de Google, forzando un *prompt* de CISO de nivel "Big Four".
- **El Fuego (Visualización):** Generación autónoma de gráficos mediante `Matplotlib` (a 300 DPI), asegurando la calidad visual necesaria para un informe corporativo
- **El Yunque (Ensamblaje):** Consolidación final mediante una plantilla rígida en **LaTeX**, utilizando el color institucional `CorpBlue` para mantener la identidad visual

[illegible]

```

nombre_proyecto = nombre_proyecto_input.replace(" ", "_")
os.makedirs(nombre_proyecto, exist_ok=True)

#CONFIGURACIÓN DEL AUTOCOMPLETADO (TABULADOR) [Esto lo hice con IA porque ni
idea] ---
def completador_rutas(texto, estado):
    coincidencias = glob.glob(texto + '*') + [None]
    return coincidencias[estado]

readline.set_completer_delims(' \t\n;')
readline.parse_and_bind("tab: complete")
readline.set_completer(completador_rutas)

#Recogemos datos
def recopilar_datos():
    parser = argparse.ArgumentParser(description='GimliForge - Red Team Report
Generator')
    parser.add_argument('-f', '--file', help='Ruta al archivo JSON (Estándar
G.I.M.L.I.)')
    args = parser.parse_args()
    #Damos opción de lanzar comandos con parámetro.
    if args.file:
        print(f"Modo Fast-Track activado. Cargando: {args.file}")
        try:
            with open(args.file, 'r', encoding='utf-8') as f:
                datos = json.load(f)
            if 'metadata' not in datos or 'hallazgos' not in datos:
                raise ValueError("Falta 'metadata' o 'hallazgos'.")
            return datos
        except Exception as e:
            print(f" Error al cargar el archivo proporcionado: {e}")
            exit(1)

    print("\n[ SELECCIONE ORIGEN DE DATOS ]")
    print("1. Cargar archivo .json existente (Estándar G.I.M.L.I.)")
    print("2. Asistente manual (Introducir hallazgos paso a paso)")
    opcion = input("Elija una opción (1 o 2): ")

    if opcion == '1':
        ruta = input("Introduce la ruta del archivo JSON: ")
        #Aquí se puede utilizar normal el TAB
        try:
            with open(ruta, 'r', encoding='utf-8') as f:
                datos = json.load(f)

```

```

        if 'metadata' not in datos or 'hallazgos' not in datos:
            raise ValueError("El JSON no cumple el estándar (Falta
'metadata' o 'hallazgos')")
        return datos
    except Exception as e:
        print(f" Error al leer el archivo: {e}")
        exit(1)

elif opcion == '2':
    print("\n--- Metadatos de la Auditoría ---")
    datos = {
        "metadata": {
            "cliente": input("Nombre del Cliente/Empresa: "),
            "auditor": input("Nombre del Auditor Jefe: "),
            "fecha_auditoria": datetime.today().strftime('%Y-%m-%d'),
            "tipo_auditoria": "Auditoría Ofensiva - Red Team"
        },
        "hallazgos": []
    }

    print("\n--- Ingesta de Hallazgos ---")
    while True:
        print(f"\n[ Vulnerabilidad #{len(datos['hallazgos'])+1} ]")
        titulo = input("Título: ")

        severidad = ""
        while severidad not in ["Crítica", "Alta", "Media", "Baja",
"Informativa"]:
            severidad = input("Severidad
(Crítica/Alta/Media/Baja/Informativa): ").capitalize()

        cvss = -1.0
        while cvss < 0.0 or cvss > 10.0:
            try:
                cvss_input = input("Puntuación CVSS (0.0 a 10.0):
").replace(',', ' ')
                cvss = float(cvss_input)
                if cvss < 0.0 or cvss > 10.0:
                    print("El CVSS debe estar entre 0.0 y 10.0")
            except ValueError:
                print("Por favor, introduce un número válido (ej. 7.5)")

        categoria = input("Categoría (ej. Aplicación Web, Red,
Criptografía...): ")

```

```

desc = input("Descripción técnica breve: ")

datos["hallazgos"].append({
    "id": f"VULN-{len(datos['hallazgos'])+1:03d}",
    "titulo": titulo,
    "severidad": severidad,
    "cvss": cvss,
    "categoria": categoria,
    "descripcion_tecnica": desc
})

continuar = input("\n¿Añadir otro hallazgo? (s/n): ").lower()
if continuar != 's':
    break
return datos
else:
    print("Opción no válida.")
    exit(1)

datos_auditoria = recopilar_datos()

# Generar los gráficos
print("\n Forjando el Dashboard Ejecutivo...")

conteo_severidad = {"Crítica": 0, "Alta": 0, "Media": 0, "Baja": 0,
                    "Informativa": 0}
conteo_categorias = {}

for hallazgo in datos_auditoria['hallazgos']:
    sev = hallazgo.get('severidad', 'Informativa')
    if sev in conteo_severidad: conteo_severidad[sev] += 1

    cat = hallazgo.get('categoria', 'Otros')
    conteo_categorias[cat] = conteo_categorias.get(cat, 0) + 1

# Rutas de salida dentro de la carpeta del proyecto
ruta_grafico_sev = os.path.join(nombre_proyecto, "grafico_severidad.png")
ruta_grafico_cat = os.path.join(nombre_proyecto, "grafico_categorias.png")

# Gráfico 1: Severidad (Rosco)
etiquetas_sev = [k for k, v in conteo_severidad.items() if v > 0]
valores_sev = [v for v in conteo_severidad.values() if v > 0]
colores_riesgo = {'Crítica': '#8B0000', 'Alta': '#FF4500', 'Media': '#FFD700',
                  'Baja': '#32CD32', 'Informativa': '#87CEEB'}

```



```

colores_usados = [colores_riesgo.get(e, '#333333') for e in etiquetas_sev]

plt.figure(figsize=(6, 6))
plt.pie(valores_sev, labels=etiquetas_sev, autopct='%1.1f%%', startangle=140,
        colors=colores_usados, shadow=True)
plt.title("Distribución del Riesgo", fontweight='bold')
plt.savefig(ruta_grafico_sev, bbox_inches='tight', dpi=300)
plt.close()

# Gráfico 2: Categorías (Barras Horizontales)
etiquetas_cat = list(conteo_categorias.keys())
valores_cat = list(conteo_categorias.values())

plt.figure(figsize=(7, 5))
plt.barh(etiquetas_cat, valores_cat, color='#2C3E50')
plt.xlabel("Número de Hallazgos")
plt.title("Vulnerabilidades por Categoría Operativa", fontweight='bold')
plt.gca().invert_yaxis()
plt.savefig(ruta_grafico_cat, bbox_inches='tight', dpi=300)
plt.close()

print(f"Gráficos generados y guardados en la carpeta '{nombre_proyecto}'.")

# --- 6. CEREBRO IA (PROMPT ULTRA-EJECUTIVO CORREGIDO) ---
print("Redacción de reporte")
datos_texto = json.dumps(datos_auditoria, indent=2, ensure_ascii=False)

prompt = f"""
Actúa como un Director de Ciberseguridad de una consultora 'Big Four'.
Redacta las secciones de un informe ejecutivo basado en esta auditoría:

DATOS: {datos_texto}

REGLAS DE ESTILO Y RESTRICCIONES OBLIGATORIAS:
1. Lenguaje de negocios: Habla de "Exposición al riesgo", "Impacto en la
continuidad de negocio", "Daño reputacional" y "Protección de activos".
2. Tono impersonal, formal y de autoridad.
3. RESTRICCIÓN CRÍTICA: ESTÁ TOTALMENTE PROHIBIDO INVENTAR O ESTIMAR CIFRAS
ECONÓMICAS CONCRETAS. NUNCA hables de cantidades de dinero o pérdidas en
euros/dólares. Límitate a describir el impacto conceptual.
4. CORRECCIÓN DE FORMATO OBLIGATORIA: Para el listado de hallazgos, usa
estrictamente el entorno \begin{itemize} y \item estándar de LaTeX. Está
COMPLETAMENTE PROHIBIDO incluir parámetros opcionales entre corchetes (ej. NO
generes \begin{itemize}[leftmargin=*) ni etiquetas personalizadas).

```

5. Solo devuelve código LaTeX puro, sin introducciones. Usa el separador exacto: ---SECCION---

Parte 1: `\section{{Resumen Ejecutivo y Exposición al Riesgo}}` (Redacta dos párrafos para la junta directiva evaluando el estado general de la seguridad).

---SECCION---

Parte 2: `\section{{Detalle Técnico de Hallazgos}}` (Genera la lista con `\begin{{itemize}}` básico. En cada ítem, pon en negrita el título, especifica el CVSS y detalla el impacto operativo).

---SECCION---

Parte 3: `\section{{Hoja de Ruta de Mitigación}}` (Redacta recomendaciones estratégicas a corto y medio plazo).

"""

try:

 response = client.models.generate_content(model='gemini-2.5-flash',
 contents=prompt)

 partes = response.text.split('---SECCION---')

 if len(partes) != 3:

 raise ValueError("Error de parsing: La IA no respetó los
delimitadores.")

except Exception as e:

 print(f" Error en la generación IA: {e}")

 exit(1)

--- 7. ENSAMBLAJE LATEX (PLANTILLA PREMIUM) ---

print("Ensamblando documento PDF final...")

plantilla_latex = r"""

\documentclass[11pt, a4paper]{article}

\usepackage[utf8]{inputenc}

\usepackage[spanish]{babel}

\usepackage{geometry}

\usepackage{graphicx}

\usepackage{xcolor}

\usepackage{titlesec}

\usepackage{parskip}

\usepackage{hyperref}

\geometry{margin=2.5cm}

\definecolor{CorpBlue}{HTML}{1A365D}

\titleformat{\section}{\Large\bfseries\color{CorpBlue}}{\thesection}{1em}{}[\titledrule]

\hypersetup{colorlinks=true, linkcolor=CorpBlue, urlcolor=CorpBlue}

```

\begin{document}

\begin{titlepage}
  \centering
  \vspace*{4cm}
  {\Huge\bfseries\color{CorpBlue} Informe Ejecutivo de Auditoría Ofensiva\par}
  \vspace{1.5cm}
  {\Large\textbf{Cliente:} {{CLIENTE}}\par}
  \vspace{1cm}
  {\large\textbf{Fecha de Emisión:} {{FECHA}}\par}
  \vspace{1cm}
  {\large\textbf{Auditor Jefe:} {{AUDITOR}}\par}
  \vfill
  {\normalsize Confidencial - Uso Exclusivo de Dirección\par}
\end{titlepage}

\tableofcontents
\newpage

{{RESUMEN}}

\vspace{1cm}
\begin{figure}[htbp]
  \centering
  \begin{minipage}{0.48\textwidth}
    \centering
    \includegraphics[width=\linewidth]{grafico_severidad.png}
  \end{minipage}\hfill
  \begin{minipage}{0.48\textwidth}
    \centering
    \includegraphics[width=\linewidth]{grafico_categorias.png}
  \end{minipage}
  \caption{Dashboard Ejecutivo: Distribución de Severidad y Categorización de Hallazgos.}
\end{figure}
\vspace{1cm}

{{HALLAZGOS}}

{{RECOMENDACIONES}}

\end{document}
"""

```

```

doc_final = plantilla_latex.replace("{CLIENTE}",
datos_auditoria["metadata"]["cliente"])
doc_final = doc_final.replace("{AUDITOR}",
datos_auditoria["metadata"]["auditor"])
doc_final = doc_final.replace("{FECHA}",
datos_auditoria["metadata"]["fecha_auditoria"])
doc_final = doc_final.replace("{RESUMEN}", partes[0].strip())
doc_final = doc_final.replace("{HALLAZGOS}", partes[1].strip())
doc_final = doc_final.replace("{RECOMENDACIONES}", partes[2].strip())

# Guardamos el fichero .tex dentro de la carpeta del proyecto
ruta_final_tex = os.path.join(nombre_proyecto,
f"informe_{nombre_proyecto}.tex")
with open(ruta_final_tex, "w", encoding="utf-8") as f:
    f.write(doc_final)

print(f"\n Proceso completado con éxito!")
print(f"Todo el material ha sido forjado dentro de la carpeta:
./{nombre_proyecto}/")
print(f"    └─ Fichero principal: informe_{nombre_proyecto}.tex")
print(f"    └─ Gráfico 1: grafico_severidad.png")
print(f"    └─ Gráfico 2: grafico_categorias.png")

```

GimliForge implementa controles estrictos para garantizar la integridad del reporte:

- **Validación de Esquema:** El sistema lanza un ValueError si el JSON no cumple con los nodos obligatorios de "metadata" y "hallazgos" que son obligatorios para que la aplicación devuelva el resultado esperado
- **Protección Económica:** Se prohíbe explícitamente a la IA la invención o estimación de cifras financieras (euros/dólares), limitando el análisis al impacto conceptual y la continuidad de negocio (Con esto generamos también una salvaguarda, el pentester no tiene que saber la cantidad de dinero que hay en una empresa)
- **Parsing Milimétrico:** La separación de secciones mediante el delimitador ---SECCION--- asegura que la estructura del LaTeX final no se corrompa

4. Roadmap de Evolución (Futuras Mejoras)

Para futuras versiones y la consolidación del proyecto en el ámbito profesional, se han planificado las siguientes capacidades:

- **Modo de Entrada Forzosa:** Se implementará una validación más estricta donde el sistema obligará a cargar únicamente archivos JSON pre-validados, eliminando por completo la necesidad de entrada manual y reduciendo el margen de error humano.
- **Dashboard Avanzado:** Se expandirá la suite de visualización para incluir gráficos adicionales que permitan un análisis más profundo de los hallazgos y su impacto, ya que por el momento la herramienta siempre va a sacar dos.
- **Revisión de seguridad:** Le he puesto una redirección DNS desde <https://gimli.adriansr.com> a <https://adriansr.com/gimli> . Esta página, la hice hace poco y probablemente tenga huecos de seguridad, es algo a pulir.

5. Anotaciones sobre el uso de la IA en este trabajo y otros detalles

- Siendo consciente de haber recibido la información de que Carlos ha suspendido a un gran número de usuarios en otra convocatoria por hacer todo el trabajo con IA, he decidido hacer este reporte a mano (con ayuda de la IA para aclararlo y que sea lo más entendible posible, eso si). Quiero puntualizar que a pesar de que la presentación estea hecha con Inteligencia Artificial y evidentemente la página web, he tenido que tocar bastante código para pulir detalles y he aprendido mucho en el proceso de hacer esta humilde herramienta, a pesar de ver las excelentes herramientas de los compañeros de clase, creo que es merecedora de ser presentada.